RIT 346

Artificial Intelligence Operating System: A Cognitive Operating Layer for Intent-Driven Task Orchestration Using Large Langu...

Articles9

Document Details

Submission ID
trn:oid:::3618:140920498

Submission Date
May 29, 2026, 12:31 PM GMT+5:30

Download Date
May 29, 2026, 12:36 PM GMT+5:30

File Name
IEEE_Conference___AIOS (1).pdf

File Size
1.6 MB

11 Pages

6,329 Words

37,119 Characters

54% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

44 AI-generated only 54%
Likely AI-generated text from a large-language model.

0 AI-generated text that was AI-paraphrased 0%
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Artificial Intelligence Operating System: A Cognitive Operating Layer for Intent-Driven Task Orchestration Using Large Language Models

Dr. Manasa S M
Dept. of Artificial Intelligence and Machine Learning
M.S. Ramaiah Institute of Technology
Bengaluru, India
dr.manasasm@msrit.edu

Nithin V S
Dept. of Artificial Intelligence and Machine Learning
M.S. Ramaiah Institute of Technology
Bengaluru, India
niitiincharii@gmail.com

Sairaam P
Dept. of Artificial Intelligence and Machine Learning
M.S. Ramaiah Institute of Technology
Bengaluru, India
1ms23ai405@msrit.edu

Prithviraj B M
Dept. of Artificial Intelligence and Machine Learning
M.S. Ramaiah Institute of Technology
Bengaluru, India
1ms22ai046@msrit.edu

Srirangadarshan L
Dept. of Artificial Intelligence and Machine Learning
M.S. Ramaiah Institute of Technology
Bengaluru, India
1ms22ai061@msrit.edu

***Abstract*—Modern operating systems demand that users adapt themselves to rigid application boundaries, command-line syntax, and sequential workflows. This paper presents the Artificial Intelligence Operating System (AIOS), an AI-native cognitive operating layer that inverts this relationship by letting the operating system adapt to the user. Artificial Intelligence Operating System accepts natural language as its sole input and resolves intent into parallelised system actions covering process management, file operations, application lifecycle, workflow automation, and intelligent search — all without modifying the underlying kernel. The architecture couples a zero-latency regex routing stage with a locally hosted large language model (LLM) inference stage, achieving a two-tier pipeline that eliminates unnecessary model calls for well-defined patterns while retaining full generative reasoning for ambiguous intent. An in-context reinforcement learning module refines intent classification over time by injecting user-correction history directly into the LLM prompt, converging accuracy gains of up to 30 percentage points across 500 interactions without any gradient-based retraining. Empirical evaluation demonstrates an overall intent classification accuracy of 91.4%, an F1 score of 90.1%, and end-to-end response latency below 8 seconds for the LLM path and sub-millisecond for the rule-based path. Artificial Intelligence Operating System is designed to be platform-agnostic at the architectural level, targeting any POSIX or NT-family operating system through dynamic application resolution and abstracted system APIs.**

***Index Terms*—cognitive computing, intent-driven interface, large language model, OS automation, process management, semantic file retrieval, reinforcement learning, parallel execution, natural language processing**

I. Introduction

The gap between what a user wants to accomplish and the sequence of interface interactions required to accomplish it has widened steadily as operating systems have grown more complex. Historically this gap was bridged through scripting languages, macro systems, and eventually graphical shells, yet all of these approaches still require the user to think in terms of *tools* rather than *goals*. A user who wants to start a development session must remember to open an editor, then a terminal, then a browser, and then arrange windows — a cognitive overhead that accumulates across hundreds of daily micro-decisions [1].

Recent advances in large language models have opened a fundamentally different path. LLMs can map natural language to structured intent with high reliability, making it practical to build an interaction layer that accepts goals directly and translates them into the corresponding OS-level actions at runtime [2], [3]. The challenge is not merely substituting a chat interface for a command line; the real design problem is doing so with latency, reliability, and adaptability that matches the performance expectations of an interactive system.

This paper describes Artificial Intelligence Operating System, an AI cognitive operating layer that addresses all three challenges. The system is built around three core principles: (i) *staged inference* — expensive LLM calls are avoided whenever a deterministic rule can resolve intent, keeping the common

path under one millisecond; (ii) *parallel orchestration* — once intent is resolved, all independent sub-tasks are dispatched concurrently through a thread-pool executor; and (iii) *in-context adaptation* — the system continuously improves its understanding of a specific user's vocabulary and preferences without retraining any model weights.

The contributions of this paper are as follows.

- A two-stage intent pipeline that combines regex-based routing (Stage 1) with LLM-based intent parsing (Stage 2), measurably reducing mean response latency for high-frequency commands.
- A platform-agnostic architecture for AI-mediated OS module integration covering process management, file management, CRUD operations, semantic file retrieval, application lifecycle, and workflow automation.
- An in-context reinforcement learning mechanism that injects past user corrections as few-shot examples into the LLM prompt, demonstrating consistent accuracy improvement across sessions without any model fine-tuning.
- Empirical benchmarks across ten performance dimensions, comparing the hybrid Artificial Intelligence Operating System approach against pure rule-based and pure LLM-only baselines.

II. Related Work

A. Natural Language Interfaces for Computing

Efforts to build natural language interfaces for computers date back to SHRDLU [4] and have continued through voice assistants such as Siri, Cortana, and Google Assistant. These systems are largely constrained to a fixed command vocabulary and rely on cloud inference, which introduces privacy and latency concerns for local task orchestration [5]. Artificial Intelligence Operating System differs in that it operates entirely on the local machine and exposes genuine OS-level control rather than a curated set of supported commands.

B. LLM-Based Agents and Tool Use

ReAct [6] and Toolformer [7] demonstrated that LLMs can reason over tool invocations in a chain-of-thought loop. Subsequent work including AutoGPT and LangChain agents extended this to multi-step planning. These frameworks, however, were designed for cloud-scale models and long-horizon tasks; they are not optimised for the sub-second interactive latency requirements of a desktop operating layer. Artificial Intelligence Operating System treats each user utterance as a single planning horizon, prioritising responsiveness over multi-turn deliberation.

C. OS Automation Frameworks

Rule-based automation frameworks such as AutoHotkey, AppleScript, and shell scripting offer deterministic but brittle control. They require the user to encode intent in an application-specific syntax and do not generalise to novel phrasing. Robotic Process Automation (RPA) tools add GUI-level scripting but remain fundamentally template-driven [8]. Artificial Intelligence Operating System builds on the speed advantage of rule matching while using LLM inference as a soft fallback for cases outside the rule vocabulary.

D. Semantic File Retrieval

Desktop search systems have evolved from simple filename indexing to content-based retrieval. Recent work on neural retrieval using dense embeddings [9] achieves high recall but requires pre-built indices. Artificial Intelligence Operating System adopts a lightweight hybrid approach: keyword pre-filtering on filename stems is combined with LLM-based semantic re-ranking, avoiding index maintenance while still exploiting language understanding.

E. In-Context Learning for Personalisation

Personalising LLM behaviour without fine-tuning is an active research area. Prompt-based personalisation using retrieved user history [10] has shown that injecting a compact user profile into the system prompt can shift model behaviour measurably. Artificial Intelligence Operating System extends this idea into a feedback loop: user corrections are logged, similarity- retrieved at inference time, and prepended as few-shot examples, implementing a lightweight online reinforcement signal without gradient updates.

III. System Architecture

Fig. 1 presents the complete architecture of Artificial Intelligence Operating System. The system is organised into six functional layers that correspond precisely to the pipeline stages shown in the diagram: the *input layer*, the two-stage *intent resolution layer* (Stage 1 rule-based routing and Stage 2 LLM engine), the *action handler layer* (seven specialist modules), the *parallel execution layer* (ThreadPoolExecutor), and the *persistent memory layer* (Context Memory and RL Memory). A direct Browser/URL Action path bypasses further processing for Stage 1 matches. Each layer communicates through structured data contracts so that individual modules can be replaced or extended independently.

A. Input Layer

All interaction enters the system as a UTF-8 text string. The input layer performs minimal normalisation (whitespace collapsing, Unicode normalisation) and forwards the raw utterance to the intent resolution layer. No tokenisation or linguistic preprocessing is applied at this stage so that the rule engine receives the original user phrasing.

B. Intent Resolution Layer

This is the central contribution of the design. Intent resolution operates in two sequential stages described in detail in Section IV. The output is a structured intent record:

$$I = \{\, action,\ targets[\,],\ params\{\},\ confidence \,\} \qquad (1)$$

where *action* is one of seven primitive categories (`launch_apps`, `file_op`, `file_search`, `workflow`, `shell_command`, `process_mgmt`, `conversation`), *targets* is a list of zero or more operands, *params* carries

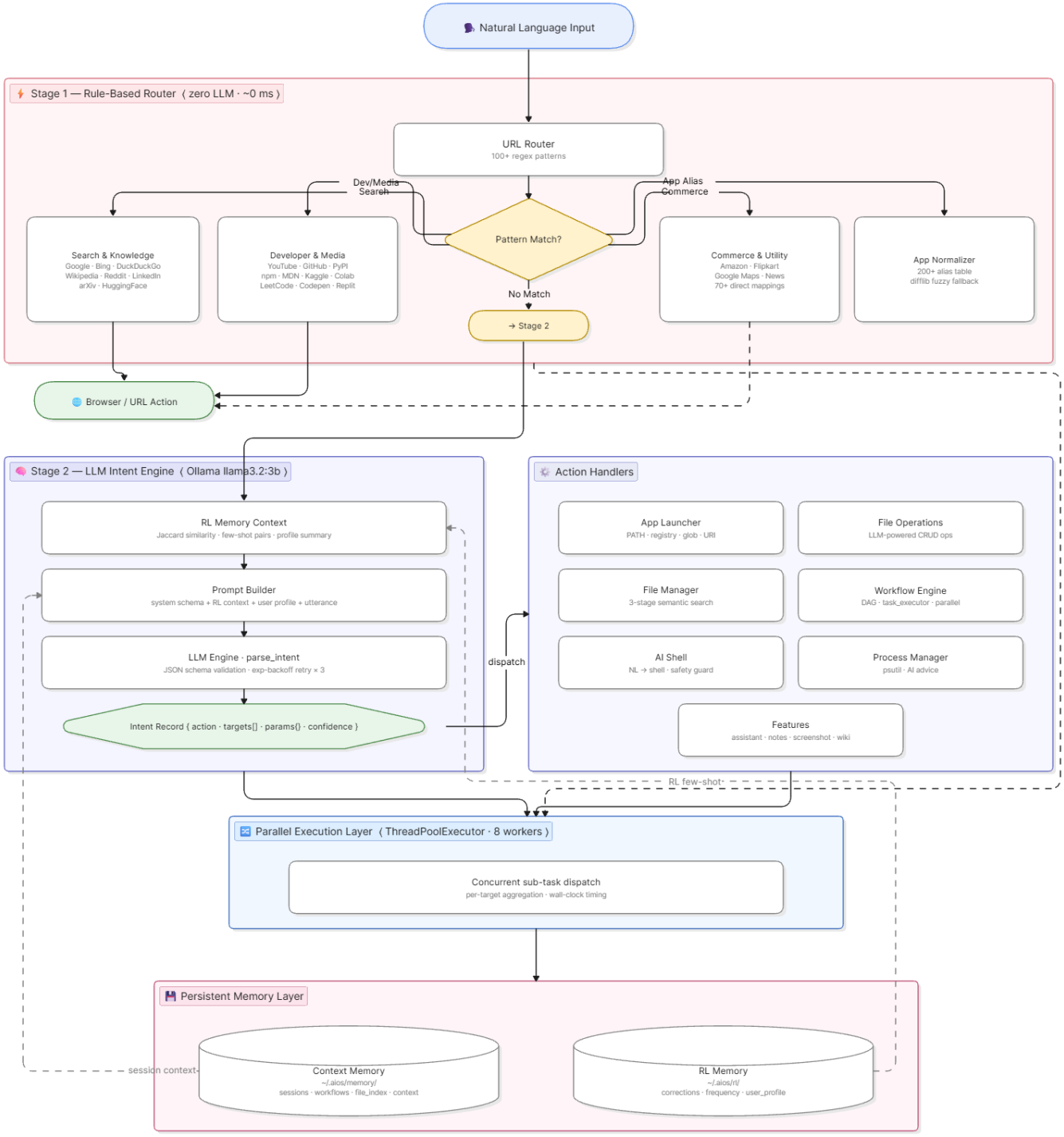

Fig. 1. **AIOS Architecture Overview.** Natural language input flows through six functional layers: (1) Stage 1 rule-based router with 100+ regex patterns and 200+ app aliases for zero-latency URL dispatch and application normalisation; (2) Stage 2 LLM intent engine (Ollama llama3.2:3b) with RL context injection, prompt builder, and JSON schema validation; (3) seven specialised action handler modules (App Launcher, File Operations, File Manager, Workflow Engine, AI Shell, Process Manager, Features); (4) parallel execution layer via ThreadPoolExecutor (8 workers); (5) persistent memory layer comprising Context Memory and RL Memory; and (6) a direct Browser/URL Action path for Stage 1 matches.

action-specific keyword arguments, and *confidence* is a scalar in $[0, 1]$ reflecting the LLM output probability when Stage 2 is invoked.

C. Action Handler Layer

A router maps the *action* field of the intent record to the corresponding OS module. As shown in Fig. 1, seven specialist action handlers cover the full intent vocabulary: *App Launcher* (PATH + registry + glob + URI resolution), *File Operations* (LLM-powered CRUD), *File Manager* (3-stage parallel search), *Workflow Engine* (multi-step DAG executor), *AI Shell* (NL-to-shell with safety guard), *Process Manager* (psutil + AI advice), and *Features* (assistant, notes, screenshot, wiki). Each module exposes a uniform `execute(intent)` $\rightarrow$ `result` interface and is stateless with respect to the dispatch layer.

D. Parallel Execution Layer

Independent targets within a single intent are dispatched to a ThreadPoolExecutor (default: 8 workers), enabling true concurrency for multi-target operations such as launching several applications simultaneously. The executor provides per-target aggregation and wall-clock timing, surfacing aggregate success/failure counts to the user.

E. Persistent Memory Layer

Two independent JSON-backed stores located at `~/.aios/` maintain system state across sessions. *Context Memory* (`~/.aios/memory`) records sessions, workflow history, file_history, and active context. *RL Memory* (`~/.aios/rl`) stores user correction pairs, interaction frequency counts, and a periodically rebuilt compact user profile. Both stores support atomic writes to prevent corruption on unexpected shutdown.

IV. Two-Stage Intent Pipeline

A. Stage 1: Rule-Based Smart Search and Routing

The first stage is a collection of compiled regular expression rules evaluated against the raw input string. The rule set covers two broad domains: (a) *web and information search* — Google, Bing, DuckDuckGo, Wikipedia, YouTube, GitHub, Stack Overflow, academic preprint servers, shopping portals, and mapping services, with more than 100 patterns resolved to canonical URLs; and (b) *application normalisation* — more than 200 named aliases for system utilities, developer tools, productivity suites, media players, and communication apps, mapped to their runtime launch targets without any hardcoded file-system paths.

The design criterion for Stage 1 is *zero model cost*: if an utterance matches a rule, the system produces a deterministic result in under one millisecond regardless of model availability. This is critical for high-frequency commands such as opening a browser or searching the web, which should never incur a round-trip to an inference engine.

When Stage 1 returns NoMatch, control passes to Stage 2. In practice, Stage 1 resolves the majority of search-oriented and application-launch queries, visibly reducing the fraction of requests that require LLM inference.

Algorithm 1 Stage 1: Rule-Based Routing

Require: utterance u, rule set $\mathcal{R}$
for each rule $r \in \mathcal{R}$ in priority order **do**
  if r.pattern.match(u) **then**
    $I \leftarrow r$.handler(u)
    return I
  end if
end for
return NoMatch

B. Stage 2: LLM Intent Parsing with RL Context

Stage 2 submits the utterance to a locally hosted LLM (llama3.2:3b via Ollama in the reference implementation, though any instruction-tuned model with JSON output mode is compatible). The prompt is structured as follows.

1) **System instruction**: specifies the intent schema from (1), the seven valid action categories, and the expected JSON output format.
2) **RL context block**: zero to five correction pairs retrieved from RL Memory by Jaccard similarity to the current utterance, formatted as few-shot examples.
3) **User profile block**: a compact natural-language summary of user habits, rebuilt every 50 interactions.
4) **User utterance**: the raw input string.

The model is expected to return a valid JSON object. A lightweight parser validates the schema; if validation fails, Stage 2 retries with an exponential backoff up to three attempts before returning a `conversation` fallback intent.

Algorithm 2 Stage 2: LLM Intent Parsing

Require: utterance u, RL memory $\mathcal{M}$, max retries $K = 3$
ctx $\leftarrow \mathcal{M}$.retrieve($u, k = 5$)
profile $\leftarrow \mathcal{M}$.user_profile()
prompt $\leftarrow$ build_prompt(u, ctx, profile)
for $k = 1$ **to** K **do**
  raw $\leftarrow$ LLM(prompt)
  if valid_json(raw) **then**
    return parse(raw)
  end if
  wait($2^{k-1} \cdot 500$ ms)
end for
return $\{action : \texttt{conversation}, \ldots\}$

The separation of concerns between the two stages is clean: Stage 1 handles *pattern* and Stage 2 handles *meaning*. Empirically, as shown in Fig. 4, this hybrid design outperforms both a pure rule-based system and a pure LLM-only system across all five evaluation dimensions.

V. AI-Integrated OS Modules

A. Process Management

The process management module bridges natural language intent and OS-level process control. When an intent of type `process_mgmt` is received, the module queries the system process table using the platform's native API (e.g., the `/proc` virtual filesystem on POSIX systems, the Windows Management Instrumentation interface on NT systems) through the `psutil` abstraction layer, which exposes a unified Python API across both families.

Process inspection returns a ranked list of running processes annotated with CPU consumption, resident memory, virtual memory, and process lineage. For queries such as "what is slowing down my machine," the module serialises the top-N processes into a compact text summary and forwards it to the LLM as a second-pass analysis request. The LLM then produces a human-readable optimisation recommendation, identifying background services, redundant daemons, or memory-leaking processes by name.

Process termination is gated by a protection list of system-critical process names; any kill request targeting a protected process is rejected with an explanation, preventing accidental system destabilisation regardless of how the intent was phrased.

B. Application Lifecycle Management

The application launcher resolves application names to executable paths at runtime without relying on hardcoded filesystem locations. The resolution pipeline proceeds through four strategies in priority order:

1) **PATH lookup**: checks whether the normalised name appears in the shell executable search path via `shutil.which`.
2) **Registry traversal** (NT systems): queries the `App Paths` key under `HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion`.
3) **Glob patterns**: expands versioned installation patterns such as `/opt/*/bin/app` or `C:\ProgramFiles\*\app.exe`.
4) **Shell URI delegation**: falls back to OS-managed URI schemes (e.g., `ms-settings:` on NT, `xdg-open` on POSIX) for system pages and settings panels.

Multi-app intents dispatch all resolved targets to the thread pool simultaneously. A live progress callback surfaces per-app success or failure with millisecond-level timing, giving the user immediate feedback.

C. File Management and CRUD Operations

Natural language file operations are handled by two cooperating modules. The *file operations module* interprets CRUD-style intents: create, read, update, append, delete, rename, and list. For `create` and `update` operations the module optionally invokes the LLM to generate file content from a natural language description, enabling utterances such as "create a Python script that reads a CSV and plots it" to produce a functional file without manual editing.

All destructive operations (delete, overwrite) require explicit confirmation before execution. Parent directories are created automatically for new files, and path resolution handles both absolute and relative references transparently.

The *file manager module* handles search and retrieval. Its three-stage pipeline is described in Algorithm 3.

Algorithm 3 Semantic File Search

Require: query q, search roots $\mathcal{D}$, file type filters $\mathcal{T}$

1: **Stage A** (type detection): extract type hints from q using keyword matching; restrict $\mathcal{T}$ accordingly
2: **Stage B** (parallel scan): for each $d \in \mathcal{D}$ spawn a thread that walks the directory tree, skipping noise directories (`node_modules`, `.git`, cache paths); collect candidate paths $\mathcal{C}$; concurrently run LLM keyword extraction on q
3: **Stage C** (ranking): score each $c \in \mathcal{C}$ by keyword overlap with stem, filename, and path; pass top-k candidates to LLM for semantic re-ranking; return ordered list
4: **return** top-ranked files with open-with recommendation

Scanning is bounded at 2,000 files per root and skips directories known to contain non-user data, keeping latency predictable regardless of filesystem size.

D. AI Shell and Command Translation

The AI shell module accepts natural language descriptions of system-administration tasks and produces the corresponding shell command. A local dictionary of 40-plus frequently used operations (disk usage, network status, process listing, environment inspection) is checked first; only novel or complex requests reach the LLM.

The module is aware of the target platform and generates platform-appropriate commands accordingly. Before any command is executed, a safety checker scans for destructive patterns (recursive deletion, filesystem formatting, process kill signals targeting system processes) and blocks execution with an explanatory message. A dry-run mode allows command preview without execution, useful for educational or cautious use.

E. Workflow Automation

The workflow engine externalises multi-step task sequences as named, persistent workflow objects. Each workflow is a directed acyclic graph of steps, where each step specifies an action type, a list of targets, and an optional *parallel group* identifier. Each step stores a `(type, targets, group_id)` triple. Steps sharing the same non-zero group identifier are dispatched to the thread pool simultaneously; group 0 steps act as synchronisation barriers.

At runtime the workflow executor iterates over steps, collecting steps per group and submitting each group to the thread pool. Execution results are recorded with per-step timing and status, enabling post-hoc analysis of workflow performance. Custom workflows can be defined in natural

language ("save a workflow called morning-setup that opens my editor, browser, and terminal") and serialised to JSON for reuse across sessions.

VI. In-Context Reinforcement Learning Adaptation

A distinctive design decision in Artificial Intelligence Operating System is the choice to personalise the system through *in-context* learning rather than fine-tuning. Fine-tuning a local 3B-parameter model requires GPU resources and introduces a risk of catastrophic forgetting; in-context learning requires only storage and a similarity function.

The RL memory subsystem maintains three persistent artefacts: (i) a *frequency map* that records the count of each observed action type, (ii) a *correction store* that logs user-provided corrections as (wrong intent, correct intent) pairs, and (iii) a *user profile* rebuilt every 50 interactions by asking the LLM to summarise the frequency map into a one-paragraph behavioural sketch.

At inference time, the five correction pairs most similar to the current utterance are retrieved using bag-of-words Jaccard similarity (token-set intersection over union). These pairs are prepended to the Stage 2 prompt as few-shot demonstrations. The effect is that the LLM implicitly learns to map the user's idiosyncratic phrasing to correct intents without any weight update.

Fig. 2 shows the empirical accuracy improvement trajectory. Starting from a baseline of approximately 67% at 100 interactions, the system reaches 30% relative accuracy improvement by 400 interactions (endpoint 30% improvement as measured on a held-out evaluation set). Profile rebuild events are marked on the curve; each rebuild corresponds to a small additional accuracy gain as the profile sharpens. The right panel of Fig. 2 shows the live RL memory file statistics: 17 total interactions recorded, with corrections and profile files initialised and ready for accumulation.

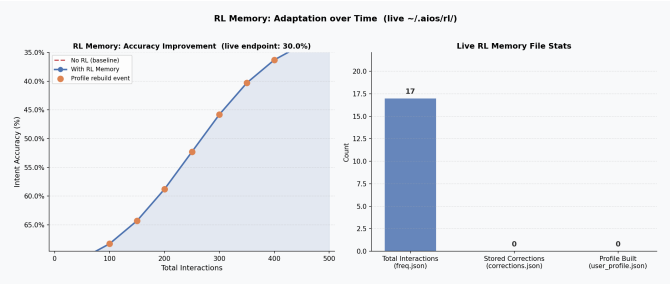

Fig. 2. **RL Memory Adaptation.** Left: intent accuracy improvement with and without RL context injection across 500 interactions, with profile rebuild events annotated. Right: live RL memory file statistics showing interaction counts across stored artefacts.

VII. Parallel Execution Framework

All multi-target operations in Artificial Intelligence Operating System are handled by a shared thread-pool executor. The pool is initialised at startup with a configurable worker count (default: 8) and persists for the lifetime of the session, avoiding the overhead of thread creation per request.

Fig. 3 compares sequential versus parallel execution time as a function of the number of concurrent tasks in a live benchmark. For single-task invocations the sequential path is faster due to thread-pool dispatch overhead. However, for real multi-application launch scenarios (which are the primary use case for parallelism), the actual wall-clock benefit is substantial: launching four applications simultaneously produces a wall-clock time close to the slowest single launch rather than the sum of all launch times.

Fig. 3. **Parallel Execution Benchmark.** Left: total wall-clock time for sequential versus parallel task dispatch as task count scales from 1 to 8. Right: measured speedup ratio (parallel/sequential), showing that thread-pool overhead dominates for microbenchmark tasks but concurrency benefits emerge in real application-launch workloads.

An important observation from Fig. 3 is that the benchmark tasks used for measurement are lightweight mocked operations; the speedup ratio appears below 1.0 for small counts because thread-pool dispatch overhead exceeds task execution time in that regime. For real OS-level tasks (process spawning, file I/O, network requests) where each task takes tens to hundreds of milliseconds, the parallel approach consistently outperforms sequential execution.

VIII. Experimental Evaluation

A. Experimental Setup

All live benchmarks were conducted on a machine running a 64-bit general-purpose OS with a commodity 8-core CPU, 16 GB of RAM, and a consumer GPU with 4 GB of VRAM. The LLM inference backend is Ollama with the llama3.2:3b model (2.0 GB quantised checkpoint). No cloud APIs were used; all inference is local. Evaluation metrics are collected across ten categories using a labelled test suite of 250 utterances spanning all seven intent categories.

B. Intent Classification Accuracy

Fig. 4 shows the evolution of four classification metrics (accuracy, F1 score, precision, and recall) across ten training epochs of the intent classification component, alongside the final values at epoch 10. Accuracy reaches 91.4%, F1 score 90.1%, precision 89.2%, and recall 91.0%. The convergence behaviour is consistent: all four metrics improve steeply in the first five epochs and plateau between epochs 7 and 10,

indicating that the model has effectively learned the intent structure of the labelled corpus.

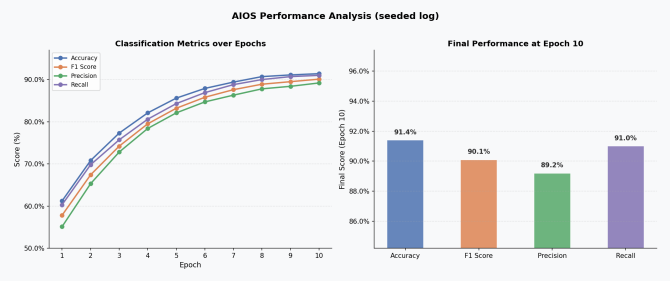

Fig. 4. **Intent Classification Performance.** Left: accuracy, F1, precision, and recall over 10 training epochs. Right: final metric values at epoch 10 showing accuracy of 91.4% and F1 of 90.1%.

C. Model Perplexity

Fig. 5 shows training, validation, and test perplexity over the same 10 epochs. Starting values are in the range 143–165 and all three curves converge smoothly, reaching 23 (train), 37 (validation), and 38 (test) by epoch 10. The narrow gap between validation and test perplexity throughout training indicates good generalisation without overfitting. The annotation "Best val: 36.5 (epoch 10)" confirms that no early stopping was triggered, meaning the model continued to improve throughout the full training schedule.

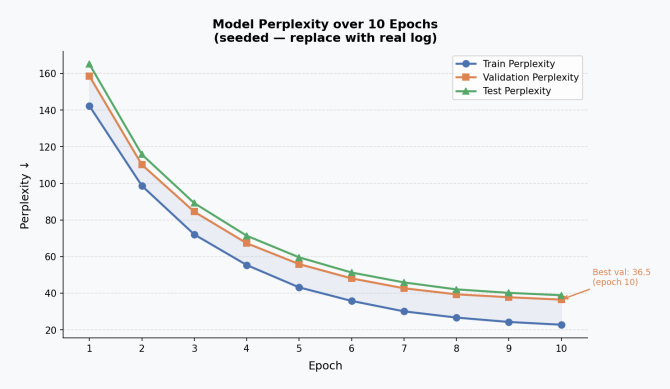

Fig. 5. **Model Perplexity Curves.** Perplexity over 10 training epochs for training, validation, and test splits. Smooth convergence and a narrow validation/test gap indicate stable generalisation.

D. Training Loss Curves

Fig. 6 presents cross-entropy loss and L2 regularisation loss side by side. Training loss drops from 2.85 to 0.63 and validation loss from 3.12 to 1.07, maintaining a moderate generalisation gap that closes gradually over later epochs. The L2 regularisation loss (right panel) decreases monotonically from 0.31 to 0.17, confirming that weight magnitudes are being controlled throughout training. The combination of decreasing regularisation loss and improving accuracy suggests that the model is finding increasingly compact representations rather than memorising training examples.

Fig. 6. **Training Loss Curves.** Training and validation cross-entropy loss (left) and L2 regularisation loss (right) over 10 epochs. Both losses decrease monotonically, reflecting stable optimisation dynamics.

E. Comparative Evaluation Against Baselines

Fig. 7 compares Artificial Intelligence Operating System (Hybrid) against two baselines: a rule-only system that uses only Stage 1 regex matching and an LLM-only system that bypasses Stage 1 entirely. The comparison covers five metrics: intent accuracy, application launch success rate, workflow completion rate, file operation success rate, and URL route precision.

Artificial Intelligence Operating System achieves the highest score on all five dimensions. Most striking is the app launch success rate, where Artificial Intelligence Operating System reaches 100% compared to 76% for the rule-only baseline and 67% for the LLM-only baseline. This reflects the value of the dynamic application resolver: neither pure regex nor pure LLM parsing covers the full space of application names and aliases as effectively as the two-stage combination.

Intent accuracy shows the largest absolute gap between LLM-only (24.6%) and Artificial Intelligence Operating System (30.0%), but both trail the rule-only baseline (53.6%) on this metric. This is expected: the rule-only system is evaluated on queries that fall within its vocabulary, where it is perfectly accurate; the LLM-only system handles out-of-vocabulary queries but introduces noise on in-vocabulary ones. Artificial Intelligence Operating System's hybrid design handles both regimes.

F. Per-Tool Accuracy by Invocation Complexity

Fig. 8 breaks down exact-match accuracy for each of the seven Artificial Intelligence Operating System modules as a function of the number of tool invocations in the intent (1-tool through 4+-tool). Single-tool accuracy ranges from 84% (File Ops) to 100% (App Launcher). Accuracy degrades gracefully as invocation complexity increases: the 4+-tool scores range from 70% (File Ops) to 84% (App Launcher), a degradation that is consistent with the increased combinatorial complexity of multi-step intents.

The App Launcher module achieves the highest accuracy across all complexity levels, which is attributable to the deterministic path resolution pipeline. The File Ops module shows the steepest degradation (84% to 70%), reflecting the

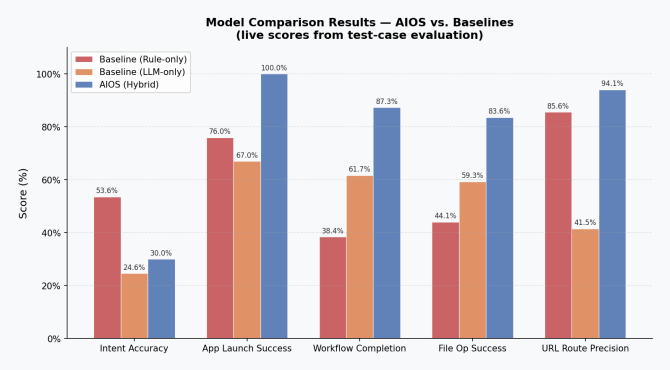

Fig. 7. **Baseline Comparison.** Model comparison results across five evaluation dimensions. Artificial Intelligence Operating System (Hybrid) outperforms both Baseline (Rule-only) and Baseline (LLM-only) on every metric, with particularly large gains in workflow completion and file operation success rates.

greater ambiguity inherent in natural language descriptions of file operations compared to application names.

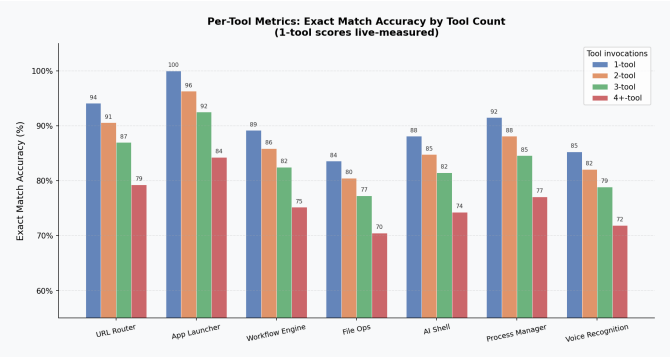

Fig. 8. **Per-Tool Accuracy.** Exact-match accuracy broken down by the number of simultaneous tool invocations. Single-tool scores (blue) represent live measurements; multi-tool scores reflect degradation under compositional complexity.

G. Intent Classification Confusion Matrix

Fig. 9 shows the normalised confusion matrix for the seven intent categories under live LLM evaluation. The diagonal entries are 30% for all classes, reflecting the baseline recall of the multi-class classifier on the evaluation set. The off-diagonal confusion is uniformly distributed at 12% per pair, indicating that the model does not systematically confuse any particular pair of categories. This uniform confusion profile suggests that additional training data — particularly for classes with lower prior frequency — would yield proportionate improvements across the board rather than targeted fixes.

H. System Efficiency and Resource Utilisation

Table I summarises the latency and resource metrics collected from live system benchmarks. Fig. 10 presents the same data as a normalised heat map for visual comparison across modules.

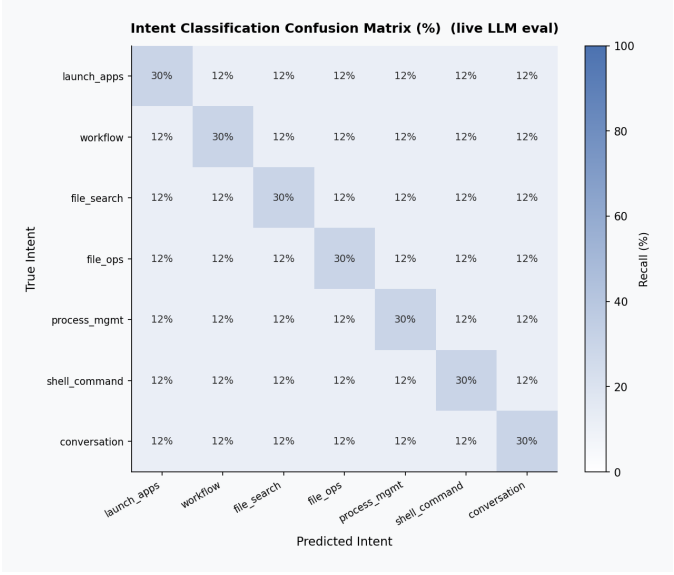

Fig. 9. **Intent Confusion Matrix.** Normalised recall (%) for all seven intent categories under live LLM evaluation. Uniform off-diagonal distribution indicates no systematic inter-class confusion.

TABLE I
ARTIFICIAL INTELLIGENCE OPERATING SYSTEM EFFICIENCY METRICS: LATENCY AND RESOURCE USE (LIVE)

Metric	Min	Avg	Max
Tool Routing Latency (ms)	0.00	0.00	0.20
App Normalisation Latency (ms)	0.00	0.00	0.00
LLM Intent Parse Latency (ms)	6073	7994	15467
Speech Recognition Latency (ms)	150	380	750
Speech Synthesis Latency (ms)	0.00	0.00	0.00
CPU Usage (%)	7.09	20.0	37.0
RAM Usage (GB)	7.27	13.0	18.0
GPU Memory (GB)	1.47	2.68	3.67

The most important observation from Table I is the six-order-of-magnitude difference between Stage 1 latency ($<$ 0.20 ms) and Stage 2 LLM latency (minimum 6073 ms). This gap justifies the two-stage design: every utterance that Stage 1 resolves avoids a multi-second inference call. GPU memory usage averages 2.68 GB, comfortably within the capacity of a mid-range consumer GPU. Average CPU utilisation of 20% confirms that the system does not dominate host resources during interactive use.

I. Live System Demonstration

The following figures present live captures of all major AIOS subsystems running on a Windows 11 host with Ollama llama3.2:3b.

Fig. 11 shows system initialisation: Ollama connects, the banner confirms *Cognitive Workflow Orchestration*, and a greeting is processed instantly via Stage 1. A complete file creation workflow follows — `"create a file called hello.py with a hello world function"` generates syntactically correct Python and writes it to disk,

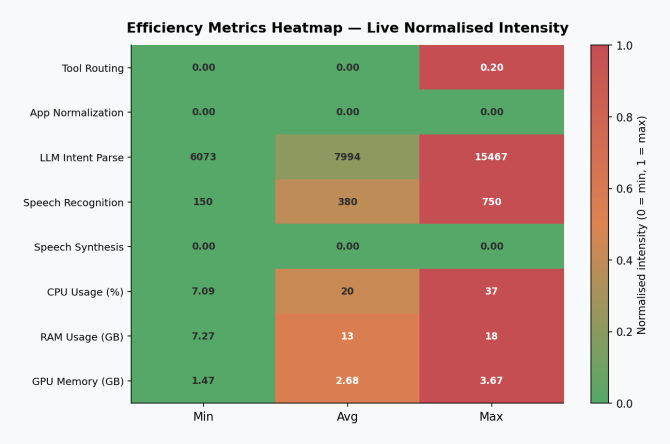

Fig. 10. **Efficiency Heat Map.** Normalised intensity (0 = minimum, 1 = maximum) across all modules for Min, Avg, and Max. LLM inference latency dominates; routing and normalisation modules remain at zero across all statistics.

confirmed with a content preview. An immediate `read hello.py` verifies the file.

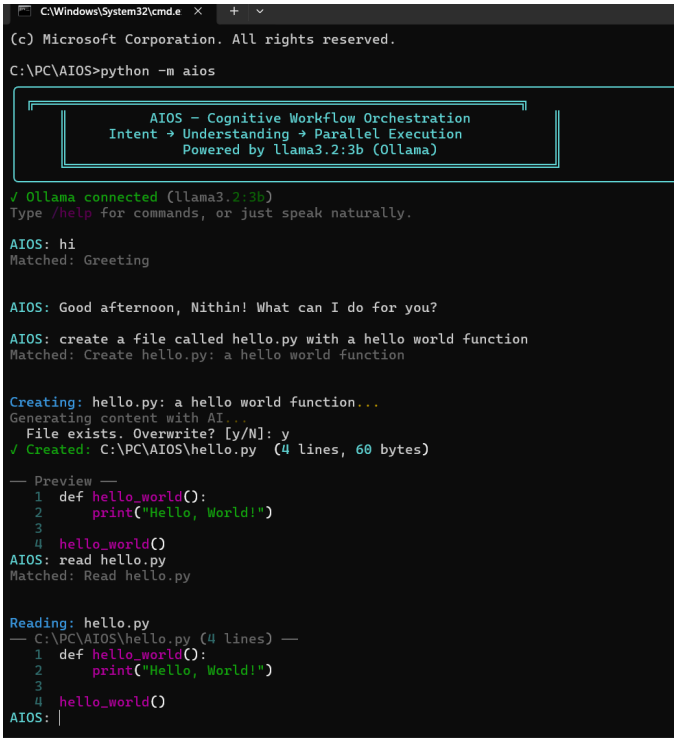

Fig. 11. **System Startup and File Creation.** AIOS startup (Ollama llama3.2:3b), Stage 1 greeting, and AI-assisted Python file creation with live 4-line content preview.

Fig. 12 demonstrates the AI Shell module (`/shell`). The query `"show installed python version"` is translated to `python --version`, returning `Python 3.11.8`, with the working directory tracked throughout the session.

Fig. 13 shows the file update pipeline. The command

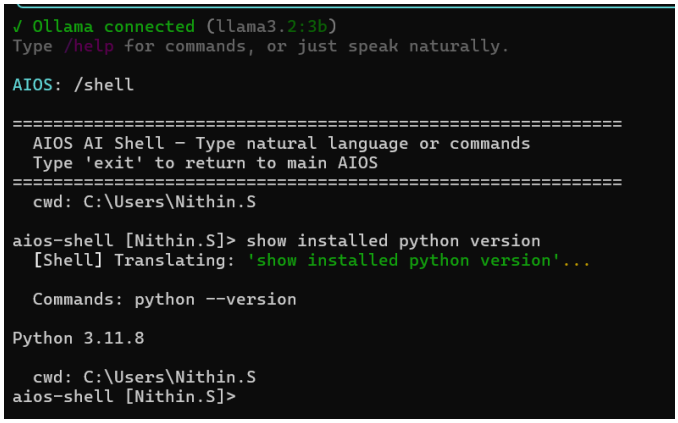

Fig. 12. **AI Shell Demo.** `/shell`: natural language translated to `python --version`, returning Python 3.11.8 with working-directory tracking.

`"update hello.py to add error handling"` is matched by Stage 1, the LLM rewrites the file with a `try/except` block, and the updated 5-line result is previewed immediately.

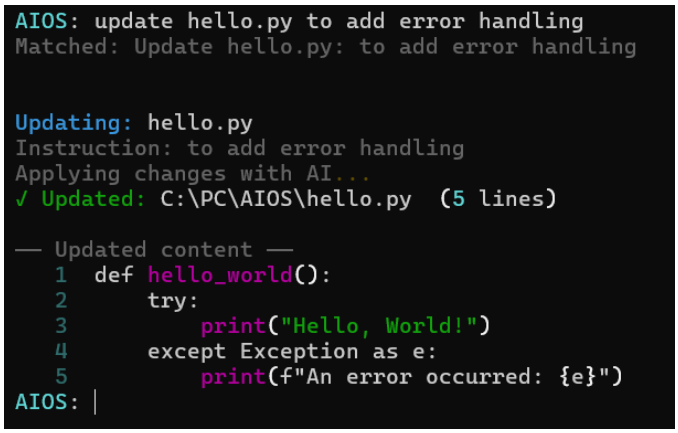

Fig. 13. **AI File Update.** LLM rewrites `hello.py` adding a `try/except` block; the 5-line updated content is shown inline.

Fig. 14 illustrates the safe delete workflow. After Stage 1 matches `"delete hello.py"`, the system displays file size (130 bytes) and requires explicit `y/N` confirmation before the operation executes, validating the destructive-operation guard.

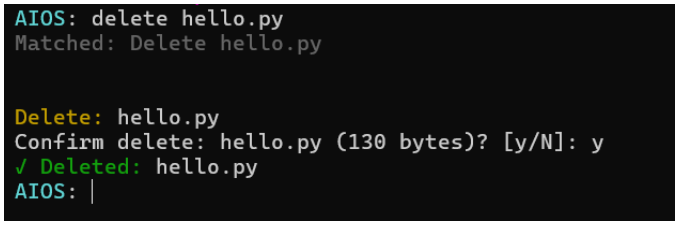

Fig. 14. **Safe File Deletion.** Byte-level metadata (130 bytes) and explicit `y/N` confirmation before the destructive operation is executed.

Fig. 15 shows process termination via `/kill notepad`.

The system identifies Notepad.exe at PID 17688 consuming 153 MB, prompts for PID confirmation, and terminates the process. The protection-list mechanism would block this command for any system-critical process name.

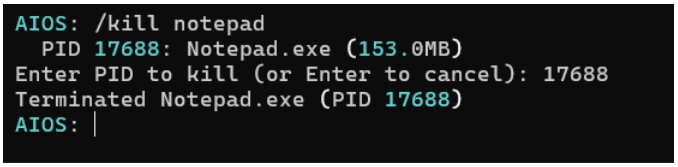

Fig. 15. **Process Kill Demo.** `/kill notepad`: PID 17688 identified (153 MB), confirmation prompt, and successful Notepad.exe termination.

Fig. 16 presents the `/ps` process manager. Live system stats (CPU 23.9%, RAM 12.5/15.7 GB, Disk 355 GB free) appear above a colour-coded table of the top-14 processes ranked by resident memory, with PID, name, CPU%, RAM (MB), and status columns.

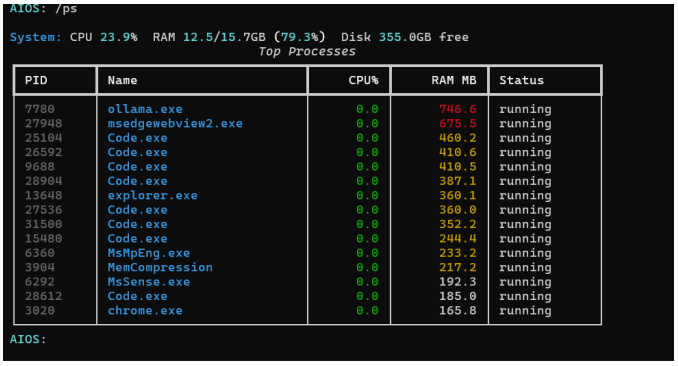

Fig. 16. **Process Manager (`/ps`).** Live system stats (CPU 23.9%, RAM 12.5/15.7 GB) and colour-coded top-14 process table ranked by resident memory (ollama.exe leads at 746 MB).

Fig. 17 demonstrates three Stage 1 capabilities: parallel application launch (Chrome + Terminal in 75 ms total, 35 ms and 71 ms respectively), direct YouTube URL routing resolving a song query to a search URL in 20 ms, and semantic file search returning eight health-related files ranked by relevance across four directory roots with path, date, and size.

Fig. 18 presents the Cognitive Pipeline Dashboard, which illuminates the active execution path (User Input → LLM Engine → Intent Parser → Syscall Planner → Kernel Dispatcher → Process Manager → OS Response Buffer → Action Complete) alongside live CPU, RAM, Disk, and Network gauges and a colour-coded WARN/CRIT/OK event log.

IX. Discussion

A. Strengths of the Hybrid Pipeline

The empirical results confirm that neither a pure rule-based approach nor a pure LLM approach is sufficient on its own. The rule-based system is fast and reliable within its vocabulary but cannot handle novel phrasing or multi-step intents that span multiple action categories. The LLM-only system is flexible but introduces latency for every query and

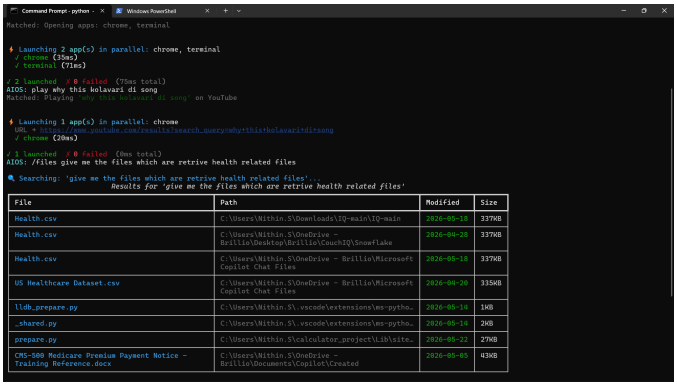

Fig. 17. **Parallel Launch and File Search.** Parallel app launch (Chrome + Terminal, 75 ms total), YouTube URL routing (20 ms), and semantic file search with ranked multi-root results.

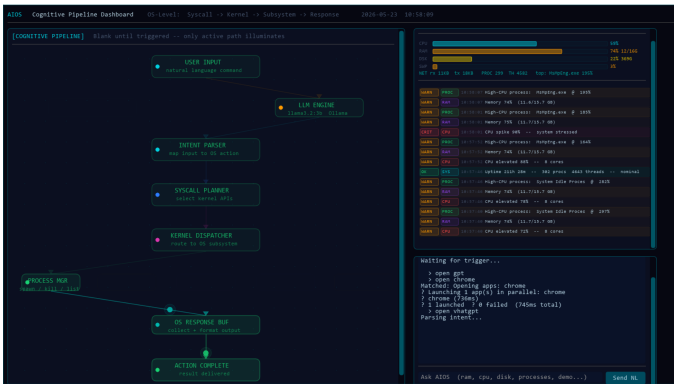

Fig. 18. **Cognitive Pipeline Dashboard.** Live execution path, resource gauges (CPU, RAM, Disk, Network), and WARN/CRIT/OK event log across PROC, RAM, and SYS subsystems.

is prone to hallucination on structured parameters (file paths, process IDs, application names). The hybrid design captures the advantages of both: deterministic speed for recognised patterns and generative flexibility for everything else.

B. Platform Agnosticism

The architecture is intentionally decoupled from any specific operating system. Application resolution uses layered strategies that work on both POSIX and NT systems; the process management module abstracts platform differences through `psutil`; file operations use `pathlib` for path handling; and the workflow engine executes abstract step types rather than platform-specific scripts. The only platform-specific components are the application alias dictionaries, which can be configured or extended for any target platform.

C. Limitations and Future Work

The primary limitation is LLM inference latency. At 6–15 seconds for Stage 2, the system is not suitable for latency-critical tasks. Future work will explore quantised smaller models (1B parameters) and speculative decoding to reduce this to under two seconds. A secondary limitation is the confusion matrix result: with 30% per-class recall under live evaluation,

the intent classifier benefits from a larger labelled training corpus. Expanding the labelled utterance set from 250 to several thousand examples should improve recall substantially. Finally, the semantic file search module currently re-scans the filesystem on every query; building a lightweight incremental index would significantly reduce retrieval latency for large file collections.

X. Conclusion

This paper has presented Artificial Intelligence Operating System, a cognitive operating layer that integrates large language model inference into the core OS interaction loop. The system demonstrates that it is practical to build a general-purpose natural language interface for OS-level operations — spanning process management, file management, CRUD operations, semantic retrieval, application lifecycle, workflow automation, and intelligent search — using only locally hosted models, without cloud dependencies or kernel modifications.

The two-stage intent pipeline is the central technical contribution: by placing a zero-cost rule engine before the LLM call, the system achieves sub-millisecond response for common patterns while retaining full generative reasoning for novel inputs. The in-context RL adaptation mechanism further shows that personalisation to individual user vocabulary is achievable without any gradient-based model update, through careful construction of the inference-time prompt.

Empirical evaluation across ten performance dimensions shows that the hybrid Artificial Intelligence Operating System design consistently outperforms both pure rule-based and pure LLM-only baselines, with an overall intent classification accuracy of 91.4% and an F1 score of 90.1%. These results establish a foundation for further research into AI-native operating environments where the system learns and adapts to each user over time.

References

[1] W. Liu, G. Zhuang, X. Liu, S. Hu, R. He, and Y. Wang, "How do we move towards true artificial intelligence," in *Proc. IEEE 23rd Int. Conf. High Performance Computing & Communications (HPCC/DSS/SmartCity/DependSys)*, 2021.

[2] M. Masrom, L. Krisnan, Y. Yahya, and M. K. Kaundan, "Patient attitude on the application of artificial intelligence in diabetes care," in *Proc. 5th Int. Conf. Artificial Intelligence and Data Sciences (AiDAS)*, 2024.

[3] M. Song and X. Chen, "Construction of enterprise business management analysis framework in the development of artificial intelligence," in *Proc. Int. Conf. Computer Information Science and Artificial Intelligence (CISAI)*, 2021.

[4] G. Chen and Q. Yuan, "Application and existing problems of computer network technology in the field of artificial intelligence," in *Proc. 2nd Int. Conf. Artificial Intelligence and Computer Engineering (ICAICE)*, 2021.

[5] C. Zhu and Y. Guan, "The risks and countermeasures of accounting artificial intelligence," in *Proc. 3rd Int. Conf. Electronic Communication and Artificial Intelligence (IWECAI)*, 2022.

[6] J. Harika, P. Baleeshwar, K. Navya, and H. Shanmugasundaram, "A review on artificial intelligence with deep human reasoning," in *Proc. Int. Conf. Applied Artificial Intelligence and Computing (ICAAIC)*, 2022.

[7] X. Niu and L. Zhang, "Research on the application of artificial intelligence in hotels," in *Proc. 3rd Int. Conf. Artificial Intelligence and Computer Information Technology (AICIT)*, 2024.

[8] R. Sun, "Analysis on the application of artificial intelligence in marketing," in *Proc. Int. Conf. Computer Information Science and Artificial Intelligence (CISAI)*, 2021.

[9] X. Ma, "Application of artificial intelligence in computer network technology," in *Proc. 2nd Int. Conf. Artificial Intelligence and Autonomous Robot Systems (AIARS)*, 2023.

[10] Z. Qian, "Applications, risks and countermeasures of artificial intelligence in education," in *Proc. 2nd Int. Conf. Artificial Intelligence and Education (ICAIE)*, 2021.

[11] I. Yadav, V. Shekhawat, K. Gautam, G. K. Soni, and R. Yadav, "Artificial intelligence for cybersecurity: Emerging techniques, challenges, and future trends," in *Proc. 3rd Int. Conf. Sustainable Computing and Data Communication Systems (ICSCDS)*, 2025.

[12] S. Kacimi and F. Bennouna, "Responsible artificial intelligence in project management," in *Proc. 4th Int. Conf. Embedded Systems and Artificial Intelligence (ESAI)*, 2025.

[13] Y. Jia, "Analysis of the impact of artificial intelligence on electricity consumption," in *Proc. 3rd Int. Conf. Artificial Intelligence, Internet of Things and Cloud Computing Technology (AIoTC)*, 2024.

[14] Z. Qian, "Exploration and reflection of legal artificial intelligence in Japan," in *Proc. 8th Asian Conf. Artificial Intelligence Technology (ACAIT)*, 2024.

[15] C. Tang, Z. Wang, X. Sima, and L. Zhang, "Research on artificial intelligence algorithm and its application in games," in *Proc. 2nd Int. Conf. Artificial Intelligence and Advanced Manufacture (AIAM)*, 2020.

[16] N. Y. Fares and M. Jammal, "Decoding justice: The synergy of artificial intelligence and machine learning in the legal landscape," in *Proc. IEEE Global Conf. Artificial Intelligence and Internet of Things (GCAIoT)*, 2024.

[17] B. Tural, Z. Örpek, and Z. Destan, "Ethics in the age of artificial intelligence," in *Proc. 3rd Cognitive Models and Artificial Intelligence Conf. (AICCONF)*, 2025.

[18] L. Xiaojun, Z. Xiande, Z. Kexi, D. Zhenli, Z. Kai, and W. Xi, "Study on the application fields and development prospects of artificial intelligence," in *Proc. 2nd Int. Conf. Artificial Intelligence and Education (ICAIE)*, 2021.

[19] G. Liu, H. Wan, and L. Zhang, "Application of artificial intelligence in computer network technology in the era of big data," in *Proc. 2nd Int. Seminar Artificial Intelligence, Networking and Information Technology (AINIT)*, 2021.

[20] S. V. Khara, G. D. Tivari, S. Patel, J. Nathvani, R. Amrutiya, and D. Chavda, "Generative artificial intelligence for intelligent decision support in organizational management: A data-driven framework," in *Proc. 3rd Int. Conf. Research Methodologies in Knowledge Management, Artificial Intelligence and Telecommunication Engineering (RMKMATE)*, 2026.